

第3章-SpringBoot3集成sa-Token权限框架

Sa-Token 介绍

Sa-Token 是一个轻量级 Java 权限认证框架，主要解决：登录认证、权限认证、单点登录、OAuth2.0、分布式Session会话、微服务网关鉴权 等一系列权限相关问题。

最新开发文档永远在：<https://sa-token.cc>

核心能力 登录认证：登录、登出、踢下线、多端互斥登录 RBAC 权限：角色校验、细粒度权限码校验、注解鉴权 配套：跨域、全局拦截、Redis 分布式会话、全局异常捕获

Spring Boot3集成sa-Token框架

第一步：Maven 引入依赖

```
<!-- Sa-Token 权限认证，在线文档：https://sa-token.cc -->
<dependency>
    <groupId>cn.dev33</groupId>
    <artifactId>sa-token-spring-boot3-starter</artifactId>
    <version>1.44.0</version>
</dependency>
<!-- Hutool 工具类 -->
<dependency>
    <groupId>cn.hutool</groupId>
    <artifactId>hutool-all</artifactId>
    <version>5.8.22</version>
</dependency>
```

第二步：application.properties 全局配置

```
sa-token.token-name=Authorization    # 前端请求头key
sa-token.timeout=604800                # token有效期7天，单位秒
sa-token.active-timeout=7200           # token有效期7天，单位秒
sa-token.is-concurrent=true
sa-token.is-share=false
sa-token.token-prefix=Bearer           # token前缀，格式：Bearer xxx
sa-token.is-log=true
sa-token.is-read-header=true
sa-token.is-read-param=false
```

第三步：登录逻辑。

```

//控制层登录逻辑
@PostMapping("/user/login")
public R<?> loginUser(@Valid @RequestBody RequestLogin requestLogin){

    return userService.loginUser(requestLogin.getUsername(),
        requestLogin.getPassword());
}
//控制层登出逻辑
@GetMapping("/user/logout")
public R<?> logoutUser(){
    return userService.logout();
}

```

```

//业务逻辑层登录、登出逻辑
@Service
public class UserService {

    @Autowired
    private SysUserMapper sysUserMapper;

    public R<?> loginUser(String username, String password){
        SysUser user = sysUserMapper.findUserByUsername(username);
        if(user == null){
            R.failure("用户账户不存在");
        }
        if(!user.getPassword().equals(password)){
            R.failure("用户密码不正确");
        }
        StpUtil.login(user.getId()); //登录 StpUtil.getTokenValue() 得到Token

        UserInfo userInfo = new UserInfo();
        userInfo.setId(user.getId());
        userInfo.setUsername(user.getUsername());
        userInfo.setRealName(user.getNickname());
        userInfo.setToken(StpUtil.getTokenValue());
        userInfo.setRoles(StpUtil.getRoleList());
        userInfo.setPermissions(StpUtil.getPermissionList());

        return R.success("用户登录成功", userInfo);
    }

    public R<?> logout(){
        StpUtil.logout();
        return R.success("账户成功退出");
    }
}

```

第四步：Sa-Token 两大核心配置类

1、权限适配器 SaTokenPermissionConfig（实现 StpInterface） 作用：框架自动调用，获取当前登录用户角色 / 权限。

```
/**
 * 自定义Sa-Token权限验证接口
 */

@Component
public class SaTokenPermissionConfig implements StpInterface {

    @Autowired
    private SysUserMapper sysUserMapper;

    // 获取用户拥有的角色
    @Override
    public List<String> getRoleList(Object loginId, String loginType) {
        List<String> roleCodes = new ArrayList<>();
        // 根据登录用户ID，获取用户拥有的角色
        Long userId = Long.valueOf(loginId.toString());
        roleCodes.add(sysUserMapper.getRolesByUserId(userId));
        return roleCodes;
    }

    // 获取用户拥有的权限
    @Override
    public List<String> getPermissionList(Object loginId, String loginType) {
        List<String> permissions = new ArrayList<>();
        Long userId = Long.valueOf(loginId.toString());
        String role = sysUserMapper.getRolesByUserId(userId);
        //普通用户具有哪些权限
        if (role.equals("USER")) {
            permissions.add("ticket:query");
        }
        //代理客服具有哪些权限
        if (role.equals("AGENT")) {
            permissions.add("ticket:query");
        }
        //管理员具有哪些权限
        if (role.equals("ADMIN")) {
            permissions.add("ticket:query");
        }
        return permissions;
    }
}
```

2、全局拦截器 SaTokenConfigure 禁止自定义 HandlerInterceptor 裸写 StpUtil，必须使用框架内置 SaInterceptor，底层已处理请求上下文时序，彻底解决 SaTokenContext 上下文尚未初始化

```
@Configuration
public class SaTokenConfigure implements WebMvcConfigurer {

    // 注册拦截器
    @Override
    public void addInterceptors(InterceptorRegistry registry) {
        // 注册 Sa-Token 拦截器，校验规则为 StpUtil.checkLogin() 登录校验。
        registry.addInterceptor(new SaInterceptor(handle -> StpUtil.checkLogin()))
            .addPathPatterns("/**") //对所有请进行拦截
            .excludePathPatterns("/user/login", "/api/**"); //除了登录的请求
    }
}
```

第五步：全局异常处理器 统一捕获 Sa-Token 所有认证异常，返回标准 JSON，前端友好处理

```

/**
 * 统一对异常进行处理
 */
@RestControllerAdvice    //捕获全局异常
public class GlobalExceptionHandler {

    @ExceptionHandler(ApiException.class)    //当发生ApiException异常，返回此错误信息
    public R<?> apiExceptionHandler(ApiException e){
        return R.failure(e.getMessage());
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public R<?> methodException(MethodArgumentNotValidException e){
        Map<String, String> errors = new HashMap<>();
        e.getBindingResult().getAllErrors().forEach((error)->{
            String feildName = ((FieldError)error).getField();    //校验属性
            String errorMsg = error.getDefaultMessage();
            errors.put(feildName,errorMsg);
        });
        return R.failure(401,"参数校验异常", errors);
    }

    // 处理未登录异常
    @ExceptionHandler(NotLoginException.class)
    public ResponseEntity handleNotLoginException(NotLoginException e) {
        //return R.error(401, "未登录或Token过期: " + e.getMessage(), null);
        Map<String, Object> response = new HashMap<>();
        response.put("code", 401);
        response.put("message", "无此角色权限，禁止访问");
        response.put("data", null);

        return new ResponseEntity<>(response, HttpStatus.UNAUTHORIZED);
    }

    // 处理无角色异常
    @ExceptionHandler(NotRoleException.class)
    public ResponseEntity handleNotRoleException(NotRoleException e) {

        //return R.error(403, "无此角色权限: " + e.getMessage(), null);
        Map<String, Object> response = new HashMap<>();
        response.put("code", 403);
        response.put("message", "无此角色权限，禁止访问");
        response.put("data", null);

        return new ResponseEntity<>(response, HttpStatus.FORBIDDEN);
    }

    // 处理无权限异常
    @ExceptionHandler(NotPermissionException.class)
    public ResponseEntity handleNotPermissionException(NotPermissionException e) {

```

```
//return R.error(403, "无此操作权限: " + e.getMessage(), null); //返回200状态码
Map<String, Object> response = new HashMap<>();
response.put("code", 403);
response.put("message", "权限不足, 禁止访问");
response.put("data", null);

return new ResponseEntity<>(response, HttpStatus.FORBIDDEN); //返回403状态码
}
}
```

Sa-Token 常用核心 API

Controller/Service 直接调用

```
// 1. 获取当前登录用户ID（推荐Long类型）
Long userId = StpUtil.getLoginIdAsLong();

// 2. 判断是否登录
boolean isLogin = StpUtil.isLogin();

// 3. 强制校验登录，未登录抛异常
StpUtil.checkLogin();

// 4. 角色校验，无admin角色直接拦截
StpUtil.checkRole("admin");

// 5. 细粒度权限校验
StpUtil.checkPermission("ticket:add");

// 6. 登出当前账号，销毁token
StpUtil.logout();
```

注解鉴权

```
@SaCheckPermission("ticket:query")    //判断该接口是否有查询权限
@GetMapping("/tickets/{id}")
public R<?> getTicket(@PathVariable Long id) {
    TicketVO ticketVO = ticketService.getTicketById(id);
    return R.success("成功获取工单详情", ticketVO);
}

// 必须拥有admin角色才能访问
@GetMapping("/list")
@SaCheckRole("admin")
public String ticketList() {
    return "票务列表";
}
```

接口测试